

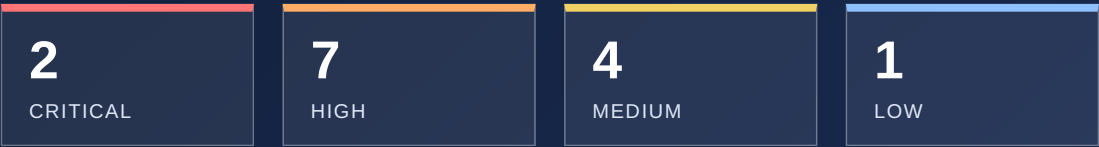
COMPLIANCE / SECURITY / ASSURANCE

Technical Compliance and SOC 2 Type 2 Readiness Audit

OVERALL VERDICT

NOT SOC 2 TYPE 2 COMPLIANT OR AUDIT-READY AS SHIPPED

The default record-only core passed functional validation. Confirmed control-design failures and missing operating evidence prevent a Type 2 claim, and the built-in self-audit is not reliable assurance evidence.



AUDIT DATE

July 22, 2026

TARGET

LPOS-v4.2.0-Complete.zip

ARCHIVE SHA-256

45d6cb6f1110c2c473b3a99f03bb22bd6d1f78dbf212deac434ff14fc23e8e69

ASSESSMENT

Point-in-time technical and SOC 2 Type 2 readiness review

No SOC 2 report or CPA opinion was supplied or issued by this review.

NAVIGATION

Contents

1. Executive decision	3
2. Why this review cannot issue SOC 2 Type 2 compliance	4
3. Scope, criteria, and limitations	5
4. Functional and integrity validation	6
5. Positive control-design observations	7
6. Confirmed findings	8
LPOS-01 - Critical: Fresh-install self-audit falsely reports SOC 2 Type 2 compliance and effectiveness	8
LPOS-03 - Critical: Agent-registered monitor service definitions can execute arbitrary shell commands as the scheduler account	9
LPOS-02 - High: Built-in control checks are structural proxies, not tests of Trust Services Criteria outcomes	10
LPOS-04 - High: Monitor configuration can read a credential file and send it to an arbitrary URL; generic checks also create SSRF exposure	11
LPOS-05 - High: Approval identity authenticity is not enforced at the runtime boundary	12
LPOS-06 - High: Dashboard has unauthenticated state-changing endpoints and no Host/Origin/CSRF boundary	13
LPOS-07 - High: Sensitive runtime and compliance state is created world-readable under the default umask	14
LPOS-08 - High: Audit and compliance evidence is not tamper-evident against the account that operates LPOS	15
LPOS-13 - High: Required organization-level control design and Type 2 operating evidence were not supplied or generated	16
LPOS-09 - Medium: Dashboard scanning escapes the configured root through symlinks and absolute kanban paths	17
LPOS-10 - Medium: Subprocess model-host boundary is not a security sandbox and output limits are enforced only after buffering	18
LPOS-11 - Medium: Release checks establish internal consistency, not publisher authenticity or provenance	19
LPOS-12 - Medium: Clean offline installation silently downgrades schema validation to JSON parsing	20
LPOS-14 - Low: Documentation and release claims have minor drift	21
7. Trust Services Criteria readiness matrix	22
8. Prioritized remediation roadmap	23
9. Evidence index	24
10. External criteria references	25
11. Final conclusion	26

1. Executive decision

Dimension	Result	Decision
Functional correctness	PASS WITH QUALIFICATIONS	Core install, test, evaluation, integrity, database, and record-only workflow paths worked in the audit environment.
Security control design	FAIL	Confirmed critical/high failures exist at monitor, approval, dashboard, local-storage, and evidence-integrity boundaries.
Built-in SOC 2 self-audit	INVALID FOR ASSURANCE	One fresh run with zero days of history was labeled compliant/effective; controls are largely structural proxies.
SOC 2 Type 2 readiness	NOT READY	Required system description, organization-level controls, and operating evidence over a defined period were not supplied or generated.
SOC 2 Type 2 report	NOT PRESENT	A licensed independent CPA must perform and issue the examination report.

Open findings: 14 total - 2 Critical, 7 High, 4 Medium, 1 Low.

Immediate operational decision

Keep the system in its bundled **record-only** external-action mode. Do not expose the dashboard remotely, do not allow agent-defined command monitor checks, do not use credentialed custom monitor URLs, and do not rely on raw `MessageIdentity` values to authorize live consequential actions. Do not represent the release as SOC 2 Type 2 compliant.

2. Why this review cannot issue SOC 2 Type 2 compliance

The AICPA Trust Services Criteria are used to evaluate controls over security, availability, processing integrity, confidentiality, and privacy. A SOC 2 examination is an assertion-based examination of the service organization's system description and relevant controls, including control design and effectiveness. A Type 2 conclusion depends on operating effectiveness over a defined period, with evidence sufficiency and timing subject to professional judgment. A SOC report is issued by a licensed, independent CPA; a software package, self-test, or this technical review cannot issue that report.

The supplied archive is therefore assessed as **readiness evidence**, not as an attestation. The release's own documentation says the same, but its machine-facing status contradicts that limitation by returning `overall: compliant` and `effective` on a fresh install.

3. Scope, criteria, and limitations

Included

- Safe archive inspection and cryptographic consistency checks.
- Clean offline installation and default record-only end-to-end flow.
- Source tests, deterministic evaluations, compilation, schema validation, coverage review, wheel RECORD, and source-to-wheel parity.
- Static review of control plane, approvals, monitor, dashboard, storage, release, installer, compliance module, and security documentation.
- Safe, local-only adversarial reproductions using temporary files, loopback HTTP servers, fabricated canary identities, and canary credentials.
- Readiness mapping to the AICPA 2017 Trust Services Criteria with revised 2022 points of focus, plus the claimed availability, confidentiality, and processing-integrity categories. Privacy was not assessed.

Not included / not supplied

- A production deployment, cloud account, IAM/SSO, endpoint controls, network architecture, real model host, real channel adapter, or live consequential adapter.
- Organization policies, personnel evidence, access reviews, vendor files, incident records, change tickets, backup job history, recovery exercises, risk-register reviews, or an observation-period population.
- A management assertion, CPA-approved SOC 2 system description, or independent service-auditor test results.
- Destructive testing, internet-target testing, denial-of-service stress testing, or use of real secrets.

Important interpretation

A missing organization-level artifact is reported as **not evidenced in the supplied scope**, not as proof that the organization has never implemented it. It remains a readiness blocker until produced and tested. The technical findings are confirmed against the supplied code and local reproductions.

4. Functional and integrity validation

Test	Result
Archive SHA-256	45d6cb6f1110c2c473b3a99f03bb22bd6d1f78dbf212deac434ff14fc23e8e69
Test environment	Linux x86_64; Python 3.13.5. The release report names Python 3.11; that exact interpreter was not available in this audit environment.
Archive safety	PASS - 271 entries; no unsafe names, symlinks, encryption, or duplicates
Release verifier	PASS - 269 immutable files, 32 specialists, 25 Standing Operations, 53 benchmarks, 17 schemas
Checksum list	PASS - 270 entries
Wheel integrity/parity	PASS - RECORD verified; 173/173 source/package files matched
Clean offline install	PASS - wheel install, pip check, init, doctor, and record-only demo
Automated tests	PASS - 196 tests and 338 subtests
Deterministic evaluations	PASS - 53/53
Schema validation	PASS in audit environment with jsonschema; QUALIFIED in clean production install because it parsed JSON only
Coverage	78% total including branches; security-sensitive monitor/checks.py 41%

Functional conclusion

The packaged default workflow is coherent and reproducible: the archive extracts safely, internal hashes are consistent, the wheel matches source, installation is offline and self-contained, the database initializes, doctor reports healthy, tests/evaluations pass, and the record-only action demo completes. This supports a **functional pass for the default local verification path**. It does not validate live adapters, authenticated channels, production monitoring, remote dashboard use, or Type 2 operating effectiveness.

5. Positive control-design observations

- Archive structure was safe to extract: 271 entries, no traversal, absolute paths, symlinks, encryption, or duplicate names.
- Release verification passed on a clean extraction over 269 immutable files; all 270 SHA256SUMS entries passed.
- Wheel RECORD verification passed (178 hashed entries) and 173 packaged source/data files matched the source tree byte-for-byte.
- Clean offline installation succeeded, pip check passed, database initialization and doctor were healthy, and the record-only end-to-end demo completed.
- All 196 tests and 338 subtests passed; all 53 deterministic evaluations passed; Python compileall passed.
- All 17 schemas passed full JSON Schema meta-validation when the optional validator was available.
- Overall test coverage was 78% including branches (80.7% statements, 68.4% branches). The monitor/checks module was only 41%, and command-line entry modules were largely untested.
- The record-only consequential-action default, exact-action canonical hashing, atomic execution claim, idempotency, single-use grants, checksummed migrations, SQLite foreign keys/WAL/FULL sync, and sandboxed file-write adapter are meaningful strengths.
- Dashboard `open_path()` resolves and rejects paths outside the root, and dashboard state uses secure temporary-file creation (mode 0600 in the audit).
- The documentation candidly states that the self-audit is not an attestation, the core is intended for a trusted single-machine deployment, privacy checks are absent, and encryption at rest is not provided. These limitations should be made consistent with machine-facing status labels.

6. Confirmed findings

LPOS-01 CRITICAL

Fresh-install self-audit falsely reports SOC 2 Type 2 compliance and effectiveness

Mapped criteria: CC4, CC5; SOC 2 system-description and operating-effectiveness requirements**Status:** Open - assurance blocker

EVIDENCE

- A fresh run produced overall="compliant", 21/21 controls "effective", days_of_history=0, and runs_in_window=21.
- Only one distinct audit timestamp existed. The 21 "runs" were 21 control rows from one execution, not 21 independent audit runs.
- effectiveness_for() labels a current passing control effective whenever its in-window ratio is ≥ 0.98 , even when the sample is one row (src/lpos_engine/compliance/audit.py:113-153).
- Overall status considers current failures only and ignores insufficient observation coverage or "not yet demonstrated" status (audit.py:226-258).
- Reproduction: evidence/reproduction-results.json -> compliance_false_positive.

Impact: The product can generate a machine-readable and human-facing assurance statement that materially overstates its status. Publishing or relying on that label could mislead customers, auditors, management, and risk owners.

REQUIRED REMEDIATION

- Rename the module and output to "control readiness monitor" or equivalent; never emit "SOC 2 compliant" or "Type 2 effective" as a self-determined result.
- Give every audit execution a unique run_id. Count distinct runs, dates, and sampling populations separately from per-control result rows.
- Make a fresh install report "not assessed / insufficient operating evidence". Require complete scope, a defined observation period, sufficient samples, and no open material exceptions before any readiness state can become "demonstrated".
- Add explicit machine-readable fields such as attestation=false, issued_by_cpa=false, and evidence_period_status=insufficient.
- Have a licensed independent CPA determine the actual Type 2 scope, period, sample sufficiency, and report conclusion.

CLOSURE / ACCEPTANCE CRITERIA

- A zero-day, one-run history cannot produce "effective" or "compliant".
- Twenty-one controls checked in one execution are reported as one run.
- Unit tests cover zero history, one run, sparse history, gaps, failures, and history tampering.
- External marketing and customer-facing output do not imply an attestation unless a valid report exists.

LPOS-03 **CRITICAL****Agent-registered monitor service definitions can execute arbitrary shell commands as the scheduler account****Mapped criteria:** CC6, CC7**Status:** Open - do not enable in a live environment**EVIDENCE**

- The inventory documentation says any agent that stands up a service appends a check definition to state/services.json (monitor/inventory.py:10-17).
- Service check dictionaries are accepted and merged without an administrative trust decision (inventory.py:106-126, 173-176).
- The command check invokes subprocess.run(..., shell=True) when command is a string (monitor/checks.py:180-200).
- A local-only reproduction registered a service with a shell string; run_audit() executed it and returned overall="ok" after the command created a marker under the scheduler identity.
- Reproduction: evidence/reproduction-results.json -> monitor.shell_command_execution.

Impact: An agent or process able to write service registration state can cross the control-plane boundary and execute commands with the monitor/scheduler account privileges. This can lead to arbitrary file access, credential theft, persistence, and full compromise of the local LPOS account.

REQUIRED REMEDIATION

- Remove shell-string command checks. Do not treat argv-only execution as sufficient if untrusted agents can still choose arbitrary executables and arguments.
- Make executable health checks admin-defined, signed or approved configuration stored outside agent-writable state. Agent registrations may reference only pre-approved check IDs and non-secret parameters.
- Run the monitor under a dedicated least-privileged account/container with a read-only filesystem view, no LPOS database write access, no general secrets, and constrained network egress.
- Apply executable and argument allowlists, resource limits, process-group termination, output limits, and immutable audit logging for configuration changes.
- Remove or similarly constrain monitor.alert.CommandTransport, which also uses a shell for string commands.

CLOSURE / ACCEPTANCE CRITERIA

- An agent-created services.json entry cannot cause any executable to run.
- Only an authenticated administrator approval can add or change executable checks.
- Negative tests cover shell metacharacters, arbitrary argv, PATH substitution, symlinks, environment inheritance, timeouts, child processes, and output flooding.

LPOS-02 HIGH

Built-in control checks are structural proxies, not tests of Trust Services Criteria outcomes

Mapped criteria: CC1-CC9, A, C, PI
Status: Open - assurance blocker

EVIDENCE

- The test control counts literal "def test_" occurrences and never executes the suite (controls.py:221-242).
- The dashboard access control reads only DEFAULT_HOST from source and does not test authentication, request origin, Host handling, CSRF, or the --host option (controls.py:270-292).
- The approval control checks only that approvals.py exists and ApprovalService is mentioned in engine.py (controls.py:295-309).
- Monitoring freshness passes when no monitor runtime exists if a catalog entry is present (controls.py:413-425).
- Availability passes when PRAGMA integrity_check text and a backup document exist; no backup job, restore result, RPO, RTO, or recovery evidence is required (controls.py:562-577).
- Release change management checks presence of a changelog, manifest, checksum list, and verifier, but not review, authorization, provenance, or execution evidence (controls.py:483-505).

Impact: The control matrix can pass even when the underlying control does not operate or does not address the criterion. This prevents the module from serving as reliable evidence for design or operating effectiveness.

REQUIRED REMEDIATION

- Replace file-presence and string-search checks with outcome-based tests tied to a documented control objective, owner, frequency, population, evidence source, and exception rule.
- Separate automated technical checks from organization-level controls that require human or third-party evidence.
- For every criterion, document why the control is sufficient, how it is tested, and what evidence is retained.
- Require independent review of the mapping and explicitly mark criteria with no implemented check as not evidenced.

CLOSURE / ACCEPTANCE CRITERIA

- Deleting or disabling the actual operating control causes the check to fail even when documentation remains.
- Control results link to immutable evidence of execution, not merely source-code presence.
- A reviewer can trace criterion -> risk -> control -> owner -> frequency -> sample -> exception -> remediation.

LPOS-04 HIGH**Monitor configuration can read a credential file and send it to an arbitrary URL; generic checks also create SSRF exposure****Mapped criteria:** CC6, CC7, C**Status:** Open - do not use credentialed custom URLs**EVIDENCE**

- github_api accepts token_file and reads any path available to the process (monitor/checks.py:146-158).
- The same check accepts an arbitrary URL and attaches Authorization: Bearer (checks.py:161-168).
- HTTP and MCP checks also accept configurable URLs without scheme, host, resolved-address, redirect, or private-network restrictions.
- A loopback capture server received Bearer AUDIT-CANARY-SECRET from a configured custom URL.
- Reproduction: evidence/reproduction-results.json -> monitor.credential_exfiltration.

Impact: A malicious or compromised configuration writer can exfiltrate files readable by the monitor, query internal services, reach metadata endpoints, or pivot through redirects and DNS behavior.

REQUIRED REMEDIATION

- Bind credentials to a fixed connector identity and allowlisted origin. Never attach a GitHub token when the URL origin differs from the approved GitHub API origin.
- Restrict token references to an admin-owned secrets directory or secret-manager handle; reject arbitrary filesystem paths and symlinks.
- Validate scheme, hostname, port, resolved IP, redirects, and every redirect target. Block loopback, link-local, multicast, metadata, and private ranges unless explicitly approved for a named connector.
- Run checks with minimal network and file privileges and redact errors/headers from state and alerts.

CLOSURE / ACCEPTANCE CRITERIA

- A custom URL cannot receive a connector credential.
- A token_file outside the approved secret root is rejected before any network request.
- Tests cover redirects, DNS rebinding/resolution changes, IPv4/IPv6 private ranges, metadata hosts, URL userinfo, alternate schemes, and symlinked token files.

LPOS-05 HIGH

Approval identity authenticity is not enforced at the runtime boundary

Mapped criteria: CC6, PI**Status:** Open - keep consequential adapters record-only

EVIDENCE

- MessageIdentity is a caller-constructible data class with channel, provider, message_id, thread_id, and sender fields (models.py:613-639).
- IdentityVerifier compares the caller-supplied sender to an allowlisted string, but does not verify a provider signature, authenticated connector, token, or attestation (approvals.py:19-52).
- LPOSRuntime.grant_action_approval accepts a caller-created MessageIdentity directly (engine.py:500-516).
- A fabricated provider/message/thread with an allowlisted sender created a grant and authorized the exact action. The bundled adapter recorded the action; the same grant path would protect a live adapter.
- Reproduction: evidence/reproduction-results.json -> approval_and_database.fabricated_approval_identity.

Impact: Exact-action hashing, expiry, replay protection, and single-use consumption are strong only after the approver is authenticated. A caller that can reach this API can impersonate the Principal by supplying the expected sender string.

REQUIRED REMEDIATION

- Introduce a trusted channel-ingestion boundary that verifies provider signatures or authenticated API sessions and issues a non-forgeable VerifiedMessage/VerifiedPrincipal assertion.
- Do not accept raw MessageIdentity from general callers for grants. Accept only an assertion minted by a registered, authenticated channel adapter and bound to raw provider-event evidence.
- Persist verification method, key/version, provider event digest, receipt time, and adapter identity; validate replay at ingestion and grant time.
- Require an end-to-end channel test before any live consequential adapter can be enabled.

CLOSURE / ACCEPTANCE CRITERIA

- Constructing MessageIdentity with an allowlisted sender is insufficient to grant approval.
- Invalid/missing provider signatures, wrong webhook secret, stale timestamps, altered event bodies, and unregistered adapters are rejected.
- A live adapter enablement gate verifies the full outbound/inbound/correlation/identity round trip described by the documentation.

LPOS-06 HIGH

Dashboard has unauthenticated state-changing endpoints and no Host/Origin/CSRF boundary

Mapped criteria: CC6, CC7

Status: Open - disable or isolate dashboard

EVIDENCE

- The HTTP handler performs GET and POST operations without authentication or authorization (dashboard/server.py:195-276).
- The body parser does not require application/json and has no maximum body size (server.py:213-225).
- Host and Origin are not checked, CSRF tokens are absent, and the CLI allows an arbitrary --host bind address (server.py:283-314).
- A POST with Host: attacker.example, Origin: <https://attacker.example>, Content-Type: text/plain, and no credential archived a project with HTTP 200.
- The response lacked common hardening headers and generic exceptions can expose internal details (server.py:227-235).
- Reproduction: evidence/reproduction-results.json -> dashboard.unauthenticated_cross_origin_mutation.

Impact: A local-network client, browser-mediated request in environments that permit it, DNS-rebinding path, or remote client when --host is broadened can read metadata and mutate dashboard state. The lack of a protocol-level trust boundary contradicts the control's "localhost-only" assurance.

REQUIRED REMEDIATION

- Keep loopback binding mandatory by default; reject non-loopback hosts unless an explicit hardened reverse-proxy mode is configured.
- Generate a high-entropy session secret, require authentication on every API route, use SameSite/HttpOnly cookies or bearer tokens, and require CSRF tokens for state changes.
- Validate Host against the actual bound host/port and enforce a strict Origin policy. Require application/json and cap request/body/query sizes.
- Add CSP, frame-ancestors/X-Frame-Options, X-Content-Type-Options, Referrer-Policy, and safe error handling. Use TLS and a real identity provider for remote use.

CLOSURE / ACCEPTANCE CRITERIA

- Unauthenticated GET/POST requests return 401/403 and do not mutate state.
- Attacker Host/Origin, text/plain forms, missing/invalid CSRF tokens, and non-loopback bind attempts are rejected.
- Security tests exercise browser-like simple requests and DNS-rebinding Host values.

LPOS-07 **HIGH****Sensitive runtime and compliance state is created world-readable under the default umask****Mapped criteria:** CC6, C**Status:** Open - confidentiality blocker on multi-user/shared hosts**EVIDENCE**

- Under default umask 022, state/lpos.db was created mode 0644. Monitor inventory/state/status and compliance history/status were also 0644.
- The database can contain prompts, context bundles, artifacts, action parameters, approvals, evidence, decisions, and audit events.
- SQLiteStore creates parent directories and the database without an explicit restrictive mode (store.py:37-52).
- The security documentation states that sensitive content may be stored and that the core provides no encryption at rest.
- Dashboard state was 0600, demonstrating a safer pattern already present elsewhere.
- Reproduction: evidence/reproduction-results.json -> approval_and_database.database_permissions and monitor.file_modes.

Impact: Other local users, backup agents, or processes with host-level read access may obtain business content and operational metadata. This is especially material when LPOS is deployed beyond the documented trusted single-user model.

REQUIRED REMEDIATION

- Create state/config directories as 0700 and files as 0600 using secure creation flags, then verify and repair permissions at startup.
- Reject or prominently warn on insecure ownership/modes. Include Windows ACL equivalents.
- Require encrypted disks or an encrypted database/storage adapter for confidentiality scope, and define key management, backup encryption, retention, and secure disposal controls.
- Redact secrets and sensitive fields from monitor errors, exports, and evidence bundles.

CLOSURE / ACCEPTANCE CRITERIA

- Fresh files are not readable by group/other regardless of process umask.
- Startup detects wrong owner/mode and fails closed or repairs with an auditable event.
- A deployment confidentiality test verifies disk, backup, export, log, and key protections.

LPOS-08 HIGH

Audit and compliance evidence is not tamper-evident against the account that operates LPOS

Mapped criteria: CC4, CC7, CC8

Status: Open - evidence reliability blocker

EVIDENCE

- SQLite update/delete triggers block ordinary event edits (sql/001_initial.sql:20-30), which is a useful accidental/ordinary-write guard.
- The same database owner can drop the trigger and alter an event. A reproduction changed an event payload after DROP TRIGGER.
- Compliance append_history() reads and rewrites the entire JSONL file; it is not append-only at the storage layer (compliance/audit.py:76-106).
- A compliance history row was removed without any integrity failure because no hash chain, signature, sequence commitment, or external copy existed.
- Reproduction: evidence/reproduction-results.json -> approval_and_database.event_tamper_resistance and compliance_false_positive.history_rewrite_demonstration.

Impact: An account compromise, malicious operator, or process with LPOS file privileges can alter the same evidence later presented as proof of control operation. Database triggers alone do not establish non-repudiation or independent evidence integrity.

REQUIRED REMEDIATION

- Hash-chain events and compliance results with monotonic sequence numbers, and sign periodic checkpoints using a key unavailable to the runtime account.
- Export events in near real time to a remote append-only/WORM destination using a write-only credential and independently monitored retention lock.
- Separate application, database-administration, evidence-custodian, and audit-reader roles in deployments that claim Type 2 assurance.
- Reconcile daily counts/digests and alert on gaps, rewrites, clock anomalies, duplicate sequences, or missing uploads.

CLOSURE / ACCEPTANCE CRITERIA

- Delete, update, insertion, reordering, truncation, and rollback of local history are detected by an independent verifier.
- The app account cannot alter retained evidence or signing keys.
- Evidence-retention and integrity checks themselves produce independently retained evidence over the audit period.

LPOS-13 HIGH**Required organization-level control design and Type 2 operating evidence were not supplied or generated**

Mapped criteria: CC1-CC9 and any scoped A/C/PI/P criteria; SOC 2 Description Criteria

Status: Open - Type 2 readiness blocker

EVIDENCE

- The supplied scope is a software release snapshot. It does not include a management assertion, complete service-organization system description, production boundary/data flows, commitments/system requirements, control owners, populations, or observation-period evidence.
- No evidence was supplied for governance oversight, personnel screening/training, joiner-mover-leaver controls, periodic access reviews, risk register review, vendor/subservice organization oversight, incident exercises, vulnerability/patch operations, change tickets/approvals, backup schedules/restore drills, retention/disposal, or exception management.
- The package includes useful technical documentation and a threat model, but those do not demonstrate that organizational controls operated consistently over a period.
- Privacy is explicitly codified without machine checks and was not evaluated in this review.

Impact: Even after technical findings are fixed, a release archive alone cannot support a SOC 2 Type 2 report. The examination concerns the service organization's described system and the design and operating effectiveness of controls, not only source code.

REQUIRED REMEDIATION

- Define the service, customers, locations, infrastructure, software, people, procedures, data classes, commitments, system requirements, subservice organizations, and complementary user-entity controls in a Description Criteria-aligned system description.
- Create a control register with criterion, risk, control, owner, frequency, population, evidence, retention, reviewer, and exception workflow.
- Implement and operate governance, IAM/JML/access review, risk management, vendor management, incident response, vulnerability management, secure change management, backup/DR, confidentiality/retention, and security-awareness controls.
- Engage an independent CPA for readiness and scope decisions, then collect evidence across the agreed examination period and remediate exceptions before the Type 2 examination.

CLOSURE / ACCEPTANCE CRITERIA

- A CPA-reviewed system description and management assertion exist for a defined scope.
- Every scoped criterion has a designed control and retained operating evidence from the agreed period.
- Exceptions have documented impact, root cause, remediation, retest, and management disposition.
- A licensed independent CPA issues the SOC 2 Type 2 report.

LPOS-09 MEDIUM**Dashboard scanning escapes the configured root through symlinks and absolute kanban paths**

Mapped criteria: CC6, C
Status: Open

EVIDENCE

- `scan_projects()` accepts directory symlinks because `entry.is_dir()` follows them (`dashboard/scanner.py:238-267`).
- `search()` walks each project path and returns matching absolute filenames (`scanner.py:270-318`).
- Kanban card path values may be arbitrary absolute paths without root containment (`scanner.py:214-225`).
- A symlinked project exposed a marker filename outside the configured root. `open_path()` separately blocks opening outside the root, which is a positive but incomplete boundary.
- Reproduction: `evidence/reproduction-results.json` -> `dashboard.scanner_root_escape`.

Impact: The dashboard can disclose filenames, paths, project metadata, and potentially traverse large/unexpected trees outside its intended scope. Impact increases if the dashboard is exposed or search results are logged/shared.

REQUIRED REMEDIATION

- Reject symlink project entries and resolve every configured root/project/card path before use.
- Require every resolved path to be inside an explicitly approved root. Treat extra roots as separately authorized, not as arbitrary path strings.
- Do not return raw absolute paths to unauthenticated clients; use opaque project/file identifiers and authorize every lookup.

CLOSURE / ACCEPTANCE CRITERIA

- Symlinks, junctions, absolute kanban paths, .. traversal, and race-time path swaps cannot expose content outside allowed roots.
- Regression tests run on supported platforms and filesystems.

LPOS-10 MEDIUM**Subprocess model-host boundary is not a security sandbox and output limits are enforced only after buffering****Mapped criteria:** CC6, CC7, C**Status:** Open before untrusted/third-party model hosts**EVIDENCE**

- The model adapter correctly uses `argv` with `shell=False` and a timeout (`adapters/subprocess_host.py:81-100`).
- The child inherits the LPOS process environment, account privileges, filesystem, working directory, and network access; no environment allowlist, OS sandbox, resource limits, or privilege separation is applied.
- `capture_output=True` buffers complete stdout/stderr before `max_stdout_bytes` is checked (`subprocess_host.py:83-100`), so the configured limit does not bound peak memory.
- Documentation calls this an isolation boundary and states a model host never receives unrestricted credential access, but the runtime does not technically enforce those properties.

Impact: A compromised or malicious configured model-host executable can read LPOS state and inherited secrets or exhaust resources. Oversized output can consume memory before the post-completion size check rejects it.

REQUIRED REMEDIATION

- Run model hosts in a dedicated low-privilege account/container/sandbox with an explicit filesystem view, no database access, constrained network egress, and a minimal environment.
- Stream stdout with a hard byte cap; terminate the process group immediately when the cap or timeout is exceeded. Apply CPU, memory, file, process, and descriptor limits.
- Use explicit `cwd`, `PATH`, executable pinning/hash, no inherited secret environment variables, and connector-specific short-lived credentials.
- Treat local/capability declarations as policy inputs that require deployment verification, not self-attested security facts.

CLOSURE / ACCEPTANCE CRITERIA

- A canary secret in the parent environment and the LPOS database are inaccessible to the child unless explicitly granted.
- Peak captured output stays under the configured limit and descendant processes are killed on timeout.
- The executable path and configuration are authenticated and change-controlled.

LPOS-11 MEDIUM**Release checks establish internal consistency, not publisher authenticity or provenance**

Mapped criteria: CC8, CC9

Status: Open before production distribution

EVIDENCE

- verify_release.py and SHA256SUMS successfully detect an unresealed file change.
- tools/reseal.py regenerates the manifest and checksum list using only data inside the tree (tools/reseal.py:1-64).
- After changing README.md and running reseal.py, the modified release passed verify_release.py.
- No signature, public key, Sigstore record, provenance/attestation, or SBOM file was present.
- The wheel RECORD and source-to-wheel hashes passed, which is useful package consistency evidence but not an external trust anchor.
- Reproduction: evidence/reproduction-results.json -> release_authenticity.

Impact: Checksums protect against accidental corruption and unsophisticated modification, but an attacker able to alter the release can regenerate all local integrity metadata and present a self-consistent malicious bundle.

REQUIRED REMEDIATION

- Sign the release manifest/digest with an offline or tightly controlled signing identity and verify it before installation.
- Publish the expected archive digest through an independent authenticated channel and implement key rotation/revocation.
- Generate an SPDX/CycloneDX SBOM and signed build provenance/attestation from a protected CI release workflow.
- Retain code review, build, test, approval, signing, and publication evidence for each release.

CLOSURE / ACCEPTANCE CRITERIA

- A locally modified and self-resealed bundle fails without access to the trusted signing identity.
- Consumers can verify archive digest, signer, provenance, source revision, build workflow, tests, and SBOM.

LPOS-12 MEDIUM**Clean offline installation silently downgrades schema validation to JSON parsing**

Mapped criteria: CC8, PI
Status: Open

EVIDENCE

- Runtime dependencies are empty; jsonschema is only in the optional dev extra (pyproject.toml:5-24).
- `_validate_schemas()` catches `ImportError` and returns a success-like status after `json.loads` only (cli.py:43-58).
- The clean offline installer reported: "JSON parsed; install the dev extra for full JSON Schema validation" and continued successfully.
- Full meta-schema validation of all 17 schemas passed in the audit environment when jsonschema was available, so this is an assurance-path weakness rather than a known bad schema.

Impact: An invalid schema can pass the production install gate if it is syntactically valid JSON. Documentation saying the installer "validates" schemas can be read more strongly than the shipped behavior.

REQUIRED REMEDIATION

- Bundle the schema validator as a runtime/offline dependency or implement an equivalent complete validation path.
- Fail installation if full validation is unavailable; do not silently convert a required gate into a warning.
- Record schema-validator name/version and validation results in release and installation evidence.

CLOSURE / ACCEPTANCE CRITERIA

- A syntactically valid but meta-schema-invalid schema makes a clean offline install fail.
- Doctor and installer report exactly the same full validation result.

LPOS-14 LOW

Documentation and release claims have minor drift

Mapped criteria: CC2, CC8
Status: Open

EVIDENCE

- docs/wiki/getting-started/first-hour.md:23 says doctor reports 21 standing operations; the release and doctor report 25.
- docs/wiki/administration/configuration.md:25 refers to SO-001 through SO-021.
- The first-hour guide says 17 "valid schemas", while a clean install performs parse-only validation unless the dev extra is installed.

Impact: Drift reduces operator confidence and demonstrates that documentation-gate checks do not validate important factual assertions.

REQUIRED REMEDIATION

- Generate counts and capability claims from release metadata in documentation tests.
- Add semantic assertions for all version/count/schema-validation statements, not only link and build checks.

CLOSURE / ACCEPTANCE CRITERIA

- All generated and hand-authored documentation matches doctor/release output and the exact validation behavior.

7. Trust Services Criteria readiness matrix

This is a readiness assessment, not an auditor opinion on effectiveness. No criterion is labeled effective because no independently tested observation-period population was supplied.

Criterion	Readiness	Basis
CC1 - Control environment	Not evidenced	Specifications and guild documents exist, but governance oversight, competence, accountability, HR, and control-owner operation were not demonstrated.
CC2 - Communication and information	Partial	Strong documentation surface; distribution, acknowledgement, escalation, and stakeholder communication evidence was not supplied. Documentation drift exists.
CC3 - Risk assessment	Partial	Threat/security models exist; no complete risk register, periodic review evidence, business/fraud/change risk process, or risk acceptance workflow was supplied.
CC4 - Monitoring activities	Deficient	Self-audit false positive, shallow checks, mutable evidence, and no independent control evaluation/deficiency process.
CC5 - Control activities	Partial	Strong tests/evaluations and deterministic guardrails, but the compliance checks do not prove control outcomes and organization-level procedures/evidence are missing.
CC6 - Logical and physical access	Deficient	Approval authenticity gap, unauthenticated dashboard, secret exfiltration path, broad process privileges, and insecure default file modes.
CC7 - System operations	Deficient	Agent-to-shell execution, SSRF/credential exposure, incomplete monitoring trust boundary, and no supplied vulnerability/incident operating evidence.
CC8 - Change management	Deficient	Checksums/changelog/tests are positive; no trusted release signature/provenance, clean install weakens schema gate, and change authorization/review evidence was not supplied.
CC9 - Risk mitigation	Not evidenced	Rollback documentation exists, but vendor/subservice organization oversight, risk mitigation decisions, and continuity operating evidence were not supplied.
A - Availability	Not demonstrated	SQLite integrity check and manual backup guidance exist; no RPO/RTO, automated backup evidence, capacity, failover, recovery exercises, or period evidence.
C - Confidentiality	Deficient	Record-only default is positive; sensitive state is plaintext and world-readable under default umask, and classification/retention/disposal controls were not demonstrated.
PI - Processing integrity	Partial	Canonical hashes, exact-action binding, idempotency, state machines, and review controls are strong; identity and monitor boundaries undermine end-to-end assurance.
P - Privacy	Not assessed	The built-in framework has no machine checks and no privacy scope, personal-information inventory, notices, consent, rights, or retention evidence was supplied.

8. Prioritized remediation roadmap

P0 - Contain immediately

- Stop publishing or relying on the built-in "SOC 2 Type 2 compliant/effective" status. Label it readiness-only.
- Disable monitor command checks and CommandTransport; reject agent-defined executable checks.
- Disable credentialed custom monitor URLs and constrain all health-check egress.
- Keep consequential action adapters record-only until authenticated channel assertions are implemented.
- Disable the dashboard or bind strictly to loopback behind authentication; do not use --host for remote exposure.
- Set state directories/files to 0700/0600 and protect the host with encrypted storage.

P1 - Engineering remediation

- Implement trusted identity attestation, dashboard auth/CSRF/Host/Origin controls, scanner containment, monitor privilege separation, and subprocess sandbox/resource limits.
- Make logs/evidence tamper-evident and export them to independently protected retention.
- Add trusted release signatures, SBOM, provenance, and a mandatory full schema-validation gate.
- Add adversarial regression tests for every confirmed reproduction; raise coverage in monitor/checks and HTTP/CLI paths.

P2 - Control design and governance

- Define the SOC 2 scope and Description Criteria system description.
- Establish control owners, policies, risk register, IAM/JML/access reviews, vendor management, incident/vulnerability management, secure SDLC/change approvals, backup/DR, confidentiality/retention, and training controls.
- Create an evidence register and exception-management process; perform an independent readiness review.

P3 - Type 2 examination readiness

- Agree scope, criteria, period, sampling, and subservice-organization treatment with a licensed independent CPA.
- Operate the controls consistently for the agreed period, retain complete evidence, remediate exceptions, and complete the CPA examination.
- Only after a report is issued should the organization communicate that it has received a SOC 2 Type 2 report, using wording consistent with that report.

9. Evidence index

Evidence	Purpose
evidence/archive-structure.json	Archive member safety, counts, sizes, and archive SHA-256.
evidence/release-verification.txt	Clean release verifier result.
evidence/sha256sum-check.txt	All SHA256SUMS entries checked.
evidence/wheel-record-verification.txt	Wheel RECORD size/digest validation.
evidence/source-wheel-parity.txt	Byte-for-byte package source/data parity.
evidence/pytest-196-summary.txt	196 tests and 338 subtests result.
evidence/coverage-output.txt and evidence/coverage.json	Coverage report and machine-readable coverage data.
evidence/evals-53.json	All 53 deterministic evaluations.
evidence/install-output.txt	Clean offline install, doctor, schema status, and record-only demo.
evidence/schema-semantic-validation.txt	Full meta-schema validation with jsonschema available.
evidence/reproduce_findings.py	Safe local-only reproduction harness.
evidence/reproduction-results.json	Machine-readable confirmed security/control failures.
evidence/static-risk-inventory.txt	Static inventory of subprocess/network/server/permission sites.
evidence/technical-evidence-manifest.json	SHA-256 manifest for evidence files.

10. External criteria references

1. AICPA & CIMA, **2017 Trust Services Criteria (With Revised Points of Focus - 2022)**, resource page dated September 30, 2023.
2. AICPA & CIMA, **2018 SOC 2 Description Criteria (With Revised Implementation Guidance - 2022)**, resource page updated July 9, 2025.
3. AICPA & CIMA, **SOC 2 Reporting on an Examination of Controls at a Service Organization Relevant to Security, Availability, Processing Integrity, Confidentiality, or Privacy**, authoritative guide description, 2022.
4. AICPA & CIMA, **SOC Logo for CPAs - Registration and Guidelines**, April 22, 2026, describing reports issued by licensed independent CPAs.
5. AICPA & CIMA, **Maintaining high standards for SOC engagements**, December 6, 2023, discussing operating effectiveness and professional judgment regarding evidence timing.
6. AICPA & CIMA, **AICPA SSAEs - currently effective**, updated April 29, 2026.

11. Final conclusion

LPOS v4.2.0 demonstrates a substantial, well-tested deterministic core and a safely constrained record-only default. It does **not** satisfy its intended security and assurance claims at several critical boundaries. The most urgent issues are the false Type 2 self-assessment, agent-to-shell monitor path, credential exfiltration/SSRF path, unverified approval identity, unauthenticated dashboard, weak local confidentiality defaults, and mutable evidence.

The release should be treated as a development/acceptance-testing system for a trusted single-user machine until P0/P1 engineering fixes are implemented. SOC 2 Type 2 readiness additionally requires organization-level control design, an AICPA Description Criteria-aligned system description, sustained operating evidence over a CPA-agreed period, and an examination/report issued by a licensed independent CPA.

Assessment disclaimer: This is an automated technical readiness assessment of the supplied artifact. It is not a SOC examination, audit opinion, certification, legal opinion, or guarantee against defects. Findings are based on the stated scope and evidence available on July 22, 2026.